PROGRAM

```
import numpy as np

# Input data
X = np.array(
   ([2, 9],
    [1, 5],
    [3, 6]),
   dtype=float
)

# Output data
y = np.array(
   ([92],
    [86],
    [89]),
   dtype=float
)

# Normalize input data
X = X / np.amax(X, axis=0)

# Normalize output data
y = y / 100

# Sigmoid Function
def sigmoid(x):
   return 1 / (1 + np.exp(-x))

# Derivative of Sigmoid Function
def derivatives_sigmoid(x):
   return x * (1 - x)

# Variable initialization
epoch = 5                # Number of training iterations
lr = 0.1                 # Learning rate

inputlayer_neurons = 2     # Number of features
hiddenlayer_neurons = 3    # Number of hidden layer neurons
output_neurons = 1         # Number of output neurons
```

```
# Weight and bias initialization
wh = np.random.uniform(
    size=(inputlayer_neurons, hiddenlayer_neurons)
)

bh = np.random.uniform(
    size=(1, hiddenlayer_neurons)
)

wout = np.random.uniform(
    size=(hiddenlayer_neurons, output_neurons)
)

bout = np.random.uniform(
    size=(1, output_neurons)
)


# Training
for i in range(epoch):

    # ----------------------------
    # Forward Propagation
    # ----------------------------

    hinp1 = np.dot(X, wh)

    hinp = hinp1 + bh

    hlayer_act = sigmoid(hinp)

    outinp1 = np.dot(hlayer_act, wout)

    outinp = outinp1 + bout

    output = sigmoid(outinp)


    # ----------------------------
    # Backpropagation
```

```
# ----------------------------

EO = y - output

outgrad = derivatives_sigmoid(output)

d_output = EO * outgrad

EH = d_output.dot(wout.T)

hiddengrad = derivatives_sigmoid(hlayer_act)

# How much hidden layer weights contributed to error
d_hiddenlayer = EH * hiddengrad


# ----------------------------
# Updating Weights
# ----------------------------

wout += hlayer_act.T.dot(d_output) * lr

wh += X.T.dot(d_hiddenlayer) * lr


# ----------------------------
# Display Output
# ----------------------------

print("----------- Epoch-", i + 1, "Starts -----------")

print("Input:")
print(X)

print("Actual Output:")
print(y)

print("Predicted Output:")
print(output)
```

```
    print("----------- Epoch-", i + 1, "Ends -----------")

    print()
```

OUTPUT

```
----------- Epoch- 1 Starts -----------
Input:
[[0.66666667 1.        ]
 [0.33333333 0.55555556]
 [1.         0.66666667]]
Actual Output:
[[0.92]
 [0.86]
 [0.89]]
Predicted Output:
[[0.84318753]
 [0.8245869 ]
 [0.84256035]]
----------- Epoch- 1 Ends -----------

----------- Epoch- 2 Starts -----------
Input:
[[0.66666667 1.        ]
 [0.33333333 0.55555556]
 [1.         0.66666667]]
Actual Output:
[[0.92]
 [0.86]
 [0.89]]
Predicted Output:
[[0.8437629 ]
 [0.82514288]
 [0.84312985]]
----------- Epoch- 2 Ends -----------

----------- Epoch- 3 Starts -----------
Input:
[[0.66666667 1.        ]
 [0.33333333 0.55555556]
 [1.         0.66666667]]
Actual Output:
[[0.92]
 [0.86]
 [0.89]]
```

```
Predicted Output:
[[0.8443289 ]
 [0.82569006]
 [0.8436901 ]]
---------- Epoch- 3 Ends ----------

---------- Epoch- 4 Starts ----------
Input:
[[0.66666667 1.        ]
 [0.33333333 0.55555556]
 [1.         0.66666667]]
Actual Output:
[[0.92]
 [0.86]
 [0.89]]
Predicted Output:
[[0.84488576]
 [0.82622867]
 [0.84424133]]
---------- Epoch- 4 Ends ----------

---------- Epoch- 5 Starts ----------
Input:
[[0.66666667 1.        ]
 [0.33333333 0.55555556]
 [1.         0.66666667]]
Actual Output:
[[0.92]
 [0.86]
 [0.89]]
Predicted Output:
[[0.8454337 ]
 [0.82675888]
 [0.84478374]]
---------- Epoch- 5 Ends ----------
```